

DreamHome Rental Management System

G2 - IT2R6

Alforque, Adrienne

Ayatal, Kyle Clarence

Diaz, Joshua

Layoung, Joshua

Application Development / Final Term Performance Innovative Task

Instructor: Jhon Harvey Babia

Bachelor of Science in Information Technology

College of Information Technology and Computing

University of Science and Technology of Southern Philippines

Lapasan, Cagayan de Oro City, 9000, Philippines

May 2026

1. Introduction

DreamHome is a rental property management system designed to help manage property records, clients, branches, staff, viewings, inspections, leases, and rental-related information. The system is divided into two main parts: a **Laravel web application** for administrative and staff-side management, and a **Flutter mobile application** for client-side access.

The Laravel web system handles the main database records and management pages, while the Flutter mobile app allows clients to register, log in, browse available properties, and request property viewings. The project uses Laravel routes, controllers, models, Blade views, and API endpoints to connect the system's front end, back end, and database. It contains both **dreamhome** for Laravel and **dreamhome_client** for Flutter.

2. Objectives

General Objective

The general objective of DreamHome is to develop a functional rental management system that allows property-related data to be stored, managed, viewed, and accessed through both a web-based Laravel system and a Flutter mobile application.

Specific Objectives

The specific objectives of the system are:

1. To provide a Laravel-based web application for managing rental property records.
2. To allow staff/admin users to manage branches, staff, owners, clients, properties, inspections, viewings, and leases.
3. To provide a Flutter mobile application where clients can register, log in, browse properties, and request property viewings.
4. To integrate the mobile application with the Laravel back end using REST API endpoints.
5. To use Laravel controllers and models to process user requests and communicate with the database.
6. To use database files, migrations, and seeders to prepare sample records for testing and demonstration.
7. To apply authentication for protected features such as client viewing requests and user-specific records.

3. Scope and Limitations

Scope

The DreamHome system covers the management of rental-related data such as properties, clients, staff, branches, owners, inspections, viewings, leases, advertisements, payments, registrations, and next of kin records. The Laravel side includes multiple controllers and models for these modules. Such as [BranchController](#), [ClientController](#), [InspectionController](#), [LeaseController](#), [OwnerController](#), [PropertyController](#), [StaffController](#), and [ViewingController](#).

The mobile application focuses on the client side. It includes client login, registration, viewing available properties, filtering properties based on max rent, requesting a viewing, and checking the client's viewing requests. The Flutter [main.dart](#) shows API calls to endpoints such as [/client/login](#), [/client/register](#), [/properties](#), [/viewings](#), and [/my-viewings](#).

Limitations

The system is mainly intended for academic demonstration and local testing. Some features may still need improvement, such as advanced search, complete deployment, full role-based mobile access, notification features, and final production-level security.

4. Target Users

Admin / Staff Users

Admin and staff users use the Laravel web application to manage the main records of the system. They can handle property listings, client information, staff records, branch records, inspection details, viewing schedules, and lease-related information.

Clients / Renters

Clients or renters use the Flutter mobile application. They can create an account, log in, browse available rental properties, check property details such as type, city, rooms, rent, and status, and submit a viewing request for a selected property. The Flutter app also allows clients to view their submitted viewing requests and check their status.

5. Tools and Technologies Used

The DreamHome system uses the following tools and technologies:

Technology / Tool	Purpose
Laravel	Main back-end framework for the web system
PHP	Server-side programming language used by Laravel
Blade	Laravel templating engine for web pages
HTML	Structure of web pages
CSS	Styling and layout of the web interface
JavaScript	Front-end interactivity where needed
Flutter	Framework used to build the mobile application
Dart	Programming language used in the Flutter application
REST API	Connects the Flutter mobile app to the Laravel back end
Laravel Sanctum / Bearer Token	Used for API authentication and protected client requests

PostgreSQL	Database storage for system records
GitHub	Repository for storing and managing project source code
Thunder Client / API Testing Tool	Used to test API endpoints, request bodies, headers, and responses
Visual Studio Code	Code editor used for development

6. System Features

Laravel Web System Features

The Laravel web system provides management features for the main DreamHome records. Based on the repository controllers and models, the system includes modules for properties, branches, staff, owners, clients, inspections, viewings, leases, payments, advertisements, registrations, and next of kin records.

Main Laravel features include:

- 1. Property Management**

Allows staff/admin users to manage rental properties, including details such as property ID, type, rooms, location, rent, owner, staff, branch, and availability/status.

- 2. Client Management**

Stores and manages client/renter information such as name, address, telephone number, email, preferred property type, max rent, and password.

- 3. Staff and Branch Management**

Allows management of branch records and staff assigned to branches.

- 4. Viewing Management**

Handles viewing requests from clients and allows staff/admin to manage viewing records.

- 5. Inspection Management**

Allows the system to record property inspection data, assigned staff, inspection dates, and comments.

6. Lease Management

Supports lease-related records between clients and properties.

7. Search and Filtering

Provides search-related functions for finding property or system records.

Flutter Mobile Application Features

The Flutter mobile application is intended for clients. It connects to the Laravel API using the base URL <http://10.0.2.2:8000/api>, which is used for Android emulator testing.

Main mobile features include:

1. Client Login

Clients log in using email and password. The app sends a POST request to the Laravel API and stores the returned token using shared preferences.

2. Client Registration

New clients can register by entering personal details, preferred property type, max rent, email, and password.

3. View Properties

Clients can browse available properties returned by the Laravel API.

4. Max Rent Filtering

The Flutter app filters properties so the client only sees properties that are within their saved maximum rent.

5. Request Viewing

Clients can submit a viewing request for a selected property using an authenticated API request with a Bearer token.

6. My Viewings

Clients can view their submitted viewing requests, including status, viewing date, property details, and assigned staff.

7. Database Design

The DreamHome database stores the records needed for rental property management. Based on the models shown in the repository, the system includes data entities such as **Advertisement**, **Branch**, **Client**, **Inspection**, **Lease**, **NextOfKin**, **Owner**, **Payment**, **Property**, **Registration**, **Staff**, **User**, and **Viewing**

Main Tables / Entities

Entity / Table	Description
Branch	Stores branch information such as branch ID, address, city, and contact details
Staff	Stores staff information and branch assignment
Client	Stores renter/client information, preferred type, max rent, email, and password
Owner	Stores property owner information
Property	Stores rental property details such as property ID, type, rooms, rent, address, owner, staff, and branch
Viewing	Stores client viewing requests and viewing status
Inspection	Stores property inspection records, inspection date, assigned staff, and comments
Lease	Stores lease records between clients and properties
Payment	Stores payment-related information

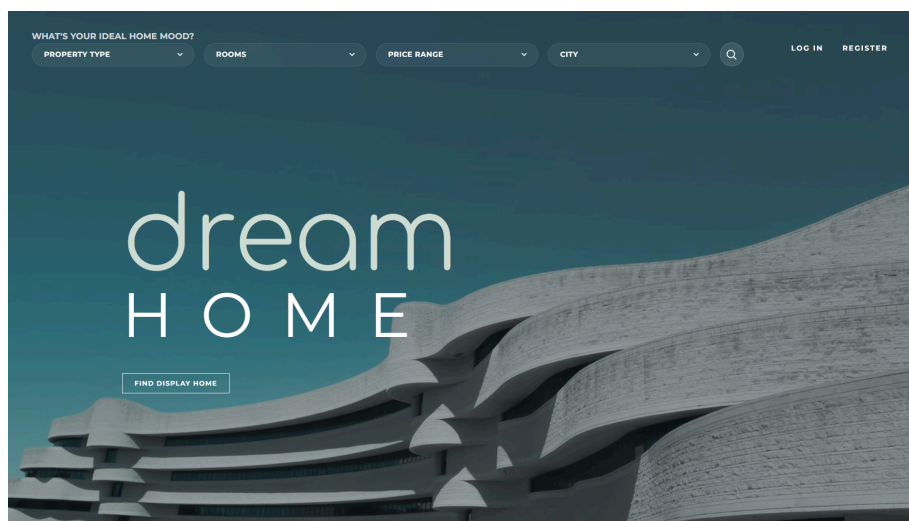
Advertisement	Stores advertisement or promotion-related records
Registration	Stores client registration records with branch and staff
NextOfKin	Stores emergency or related contact details
User	Stores Laravel authentication user records

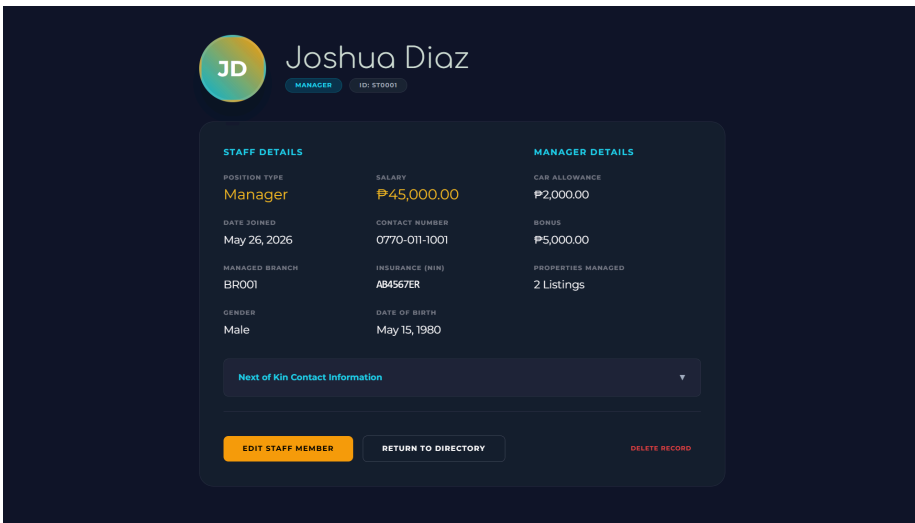
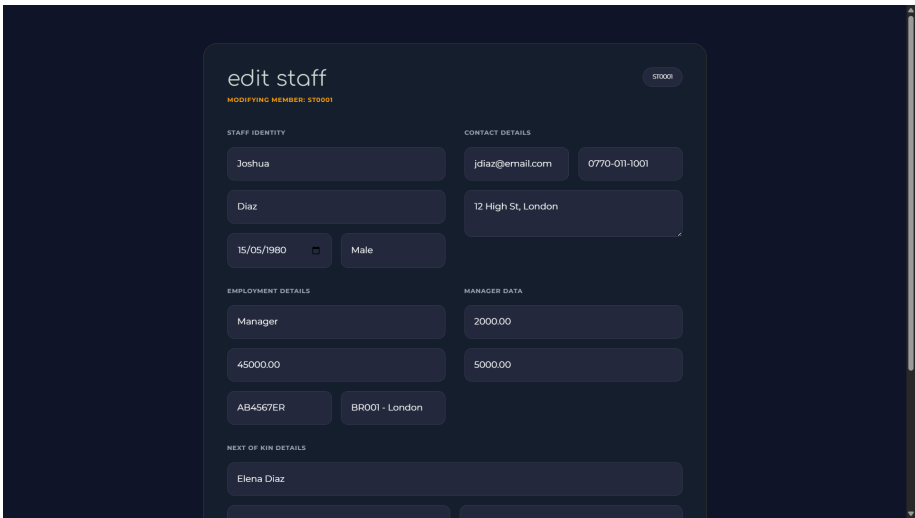
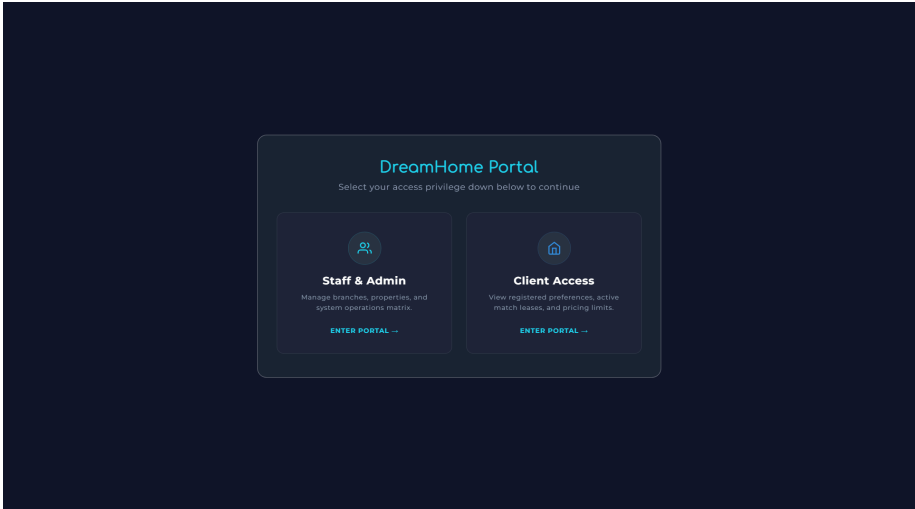
Relationships

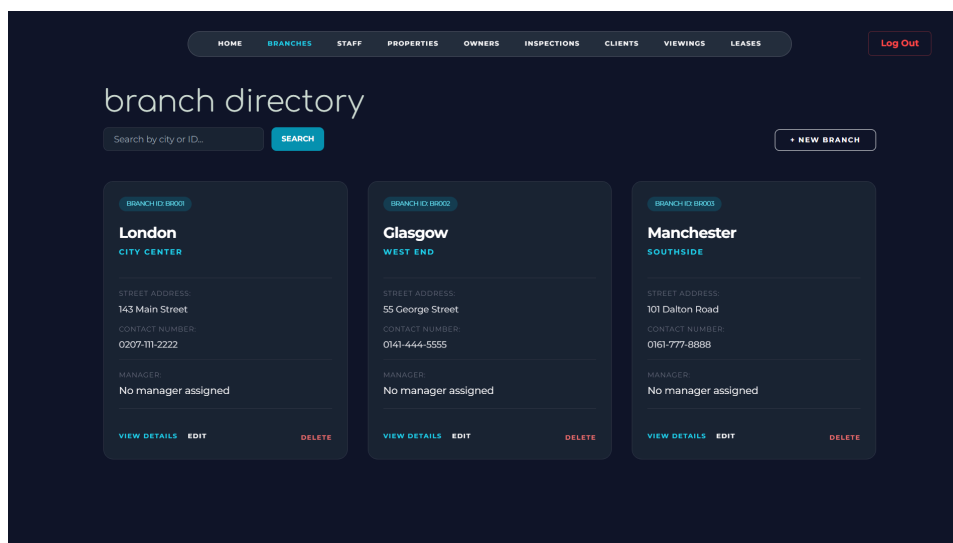
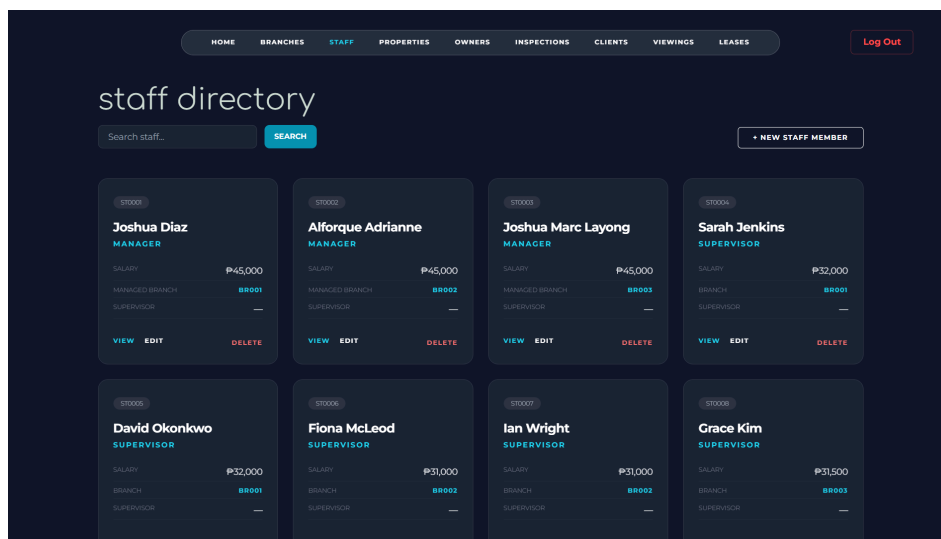
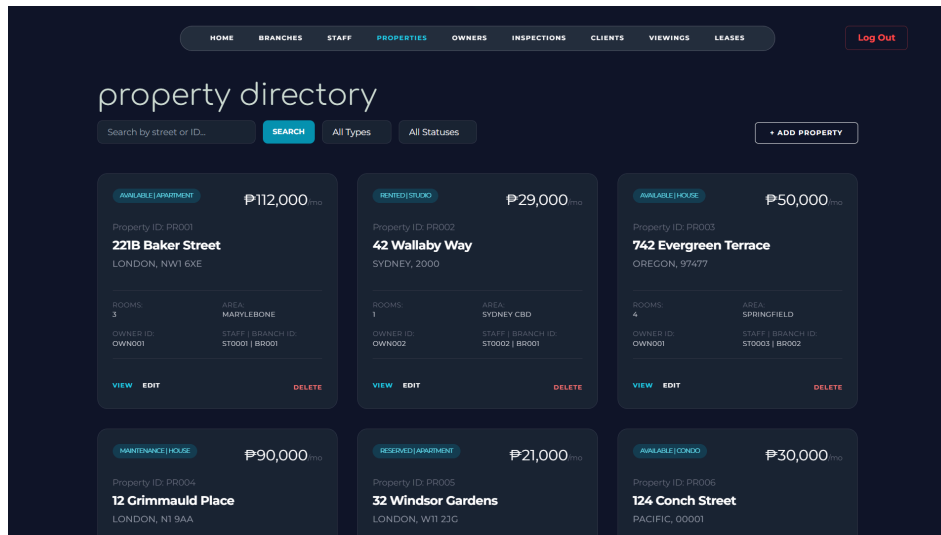
The database uses relationships between records. For example, a branch can have many staff members, a staff member can manage properties, an owner can own properties, a client can request viewings, and a property can be connected to inspections, leases, and viewing records. These relationships help the system organize rental property operations properly.

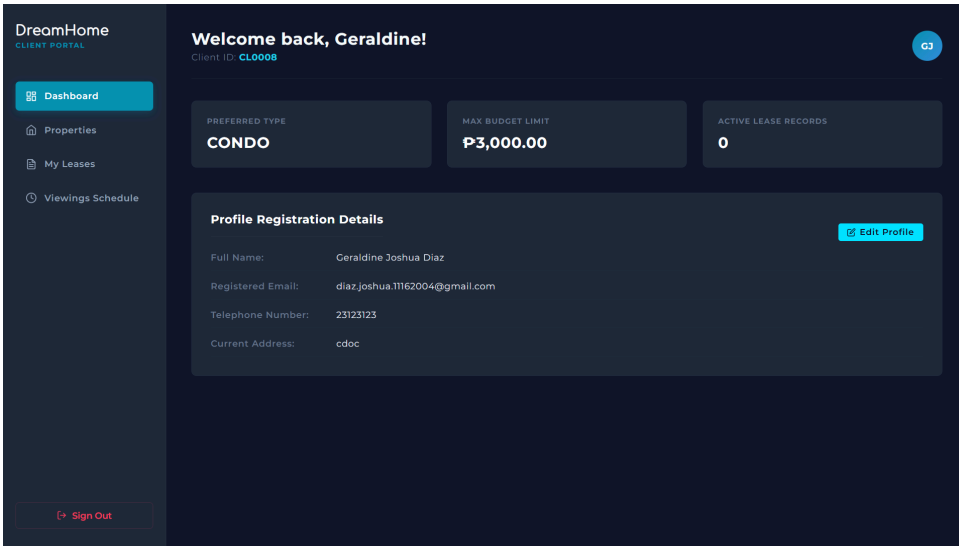
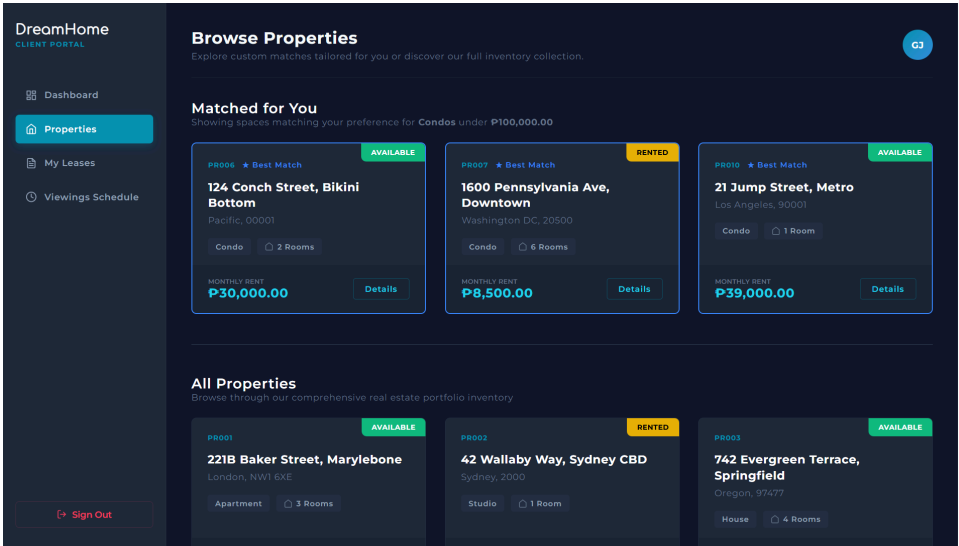
8. System Screenshots

Laravel/web/staff

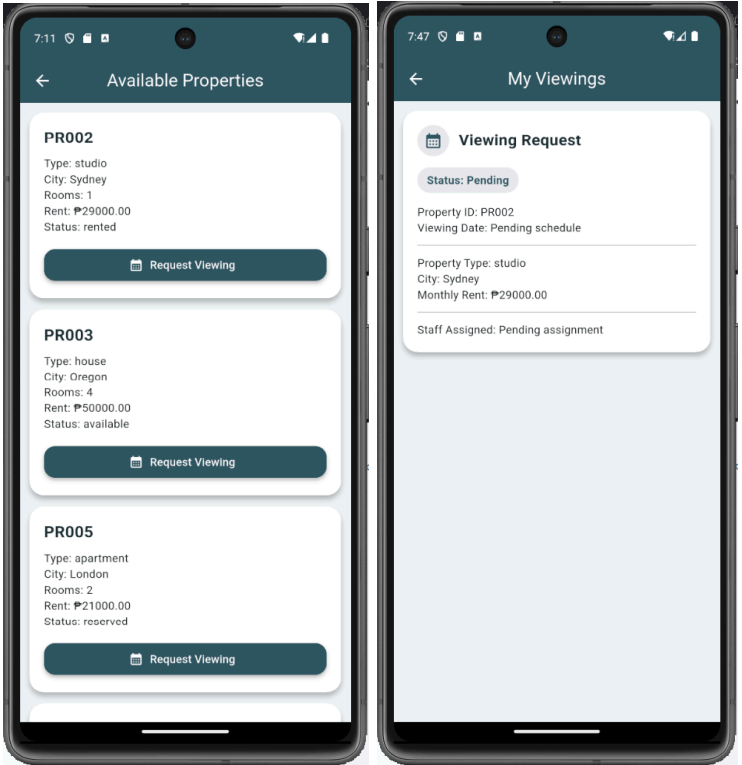
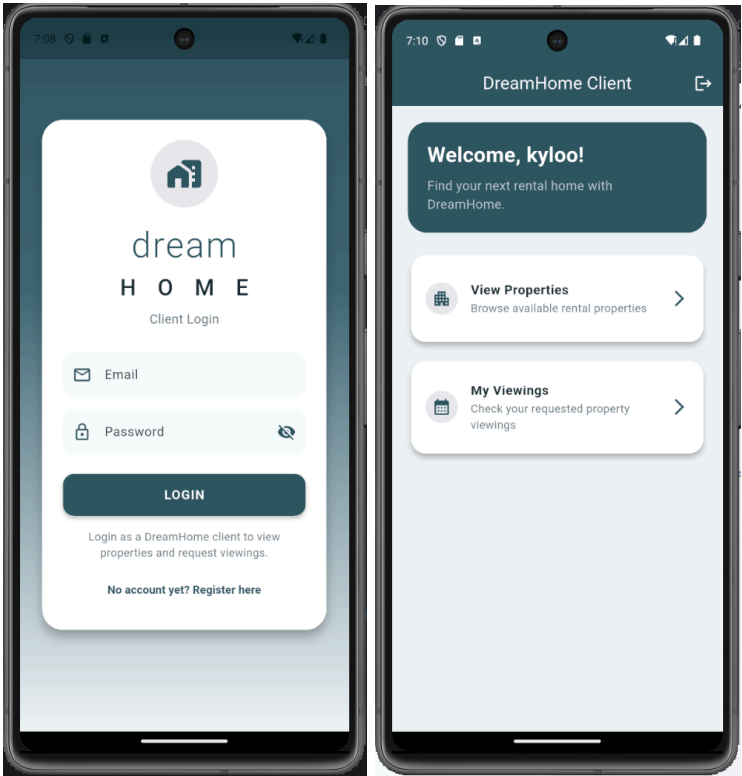








Flutter/Client



9. Source Code Explanation

The DreamHome source code is divided into two major parts: the Laravel web application and the Flutter mobile application. The repository contains the `dreamhome` folder for the Laravel project and the `dreamhome_client` folder for the Flutter client app.

Laravel Source Code

The Laravel system follows the MVC structure:

Component	Explanation
Models	Represent database tables such as Client, Property, Staff, Viewing, Inspection, and Lease
Views	Blade files used to display the web pages
Controllers	Handle user requests, validation, database operations, and page/API responses
Routes	Define web and API URLs used by the system
Migrations/SQL	Prepare and structure the database
Seeders	Insert sample records for testing and demonstration

The [routes/web.php](#) file defines web routes for public pages, dashboard access, and protected management resources. The file includes resource routes for modules like properties, branches, staff, owners, clients, inspections, viewings, and leases.

The [routes/api.php](#) file defines API-related routes used by the Flutter mobile application. The visible API route section includes routes for creating viewing requests, retrieving client viewings, and logging out.

Controller Explanation

Controller	Purpose
PropertyController	Handles property records, property listing, create, update, delete, and related staff/branch functions
ClientController	Handles client records in the Laravel web system
StaffController	Handles staff records and branch-related staff retrieval
BranchController	Handles branch records
OwnerController	Handles owner records
ViewingController	Handles viewing records on the Laravel web side
InspectionController	Handles property inspection records
LeaseController	Handles lease records

SearchController	Handles search-related pages and search results
ClientAuthController	Handles client registration, login, logout, and API authentication for the mobile app
ClientPropertyController	Handles property data returned to the mobile app
ClientViewingController	Handles mobile viewing requests and retrieval of the logged-in client's viewings

Flutter Source Code

The Flutter application is mainly contained in [dreamhome_client/lib/main.dart](#). It includes screens and functions for login, registration, dashboard, property browsing, viewing requests, and viewing history. The app uses HTTP requests to communicate with the Laravel API and shared preferences to store the login token and client data.

API Integration

The mobile app sends HTTP requests to the Laravel API. For example:

Mobile Feature	API Purpose
Login	Sends email and password to verify the client
Register	Sends client details to create a new client account
View Properties	Retrieves available property records

Request Viewing	Sends selected property ID using authenticated request
My Viewings	Retrieves the logged-in client's viewing records

10. Testing Summary

The DreamHome system was tested by checking the Laravel web pages, API endpoints, Flutter mobile application screens, and database records.

Features Tested

Feature Tested	Test Data / Action	Expected Result
Client Login	Enter registered email and password	Client successfully logs in and token is saved
Client Registration	Enter new client details	New client account is created
Property Display	Open View Properties in Flutter	Properties are loaded from the Laravel API
Max Rent Filter	Use a client max rent value	Properties above the client's max rent are not displayed
Request Viewing	Tap Request Viewing on a property	Viewing request is submitted

My Viewings	Open My Viewings screen	Client's viewing requests are displayed
Laravel CRUD	Add, view, edit, delete records	Records are properly managed
API Authentication	Send request with and without token	Protected routes only work with valid token
Database Records	Check database tables	Data is stored and retrieved correctly

API Testing

Thunder Client was used to test API requests. The tests include sending POST requests for login and registration, GET requests for properties and viewings, and authenticated requests using Bearer tokens.

11. Challenges Encountered

During the development of the DreamHome system, several challenges were encountered:

1. Laravel and Flutter API Integration

Connecting the Flutter mobile app to the Laravel API required correct API URLs, headers, request bodies, and response handling.

2. Authentication Token Handling

The system needed to store the client token after login and include it in protected API requests using the Bearer token format.

3. Database Relationship Management

The system includes many related entities such as clients, properties, staff, branches, viewings, inspections, and leases. These relationships had to be handled carefully to avoid missing or incorrect data.

4. Property Filtering

The Flutter app needed logic to hide properties with rent higher than the client's max rent.

5. Route and Controller Management

Since the Laravel project contains many modules, organizing the routes and controllers correctly was necessary to prevent errors.

Solutions Applied

To solve these problems, API routes were created, controller functions were used to process requests, Bearer token headers were added for protected requests, and Flutter HTTP requests were connected to the Laravel API. Testing was also performed using the mobile app and API testing tools.

12. Conclusion and Recommendations

The DreamHome Rental Management System successfully demonstrates an application that combines a Laravel web system, database management, REST API integration, and a Flutter mobile application. The system allows staff/admin users to manage rental-related records through the web application, while clients can use the mobile app to register, log in, view properties, request viewings, and check their viewing status.

The project meets the main requirements of having an existing Laravel web system, Flutter mobile application, login authentication, REST API integration, multiple API endpoints, and working mobile features. The system can still be improved by adding notifications, better deployment support, improved UI consistency, advanced filtering, role-based dashboards, and stronger validation.